

Structure-Aware Retrieval for Parametric Construction Catalogs Using LLM-Based Parameter Extraction

Recuperación Estructurada en Catálogos Paramétricos de Construcción
Mediante Extracción de Parámetros Basada en LLM

Abstract

Parametric catalogs—where items share a concept but differ in discrete parameter values—are common in construction, manufacturing, and e-commerce. Standard retrieval methods treat items as flat text, causing what we term *parametric collapse*: dense models achieve 96–99% concept-level accuracy but only 12–45% at the item level. We propose a three-stage pipeline that decomposes retrieval into (1) dense concept identification, (2) parameter extraction via rules or LLM slot-filling, and (3) deterministic catalog lookup. On the BC3CAT Spanish construction catalog (47,500 items, 16,590 queries), the rule-based pipeline achieves 90.3% item-level Acc@1, outperforming all neural methods and surpassing standard BM25 (86.9%). Ablation across two LLMs and five prompt strategies shows that closed-set classification benefits from model scaling while open-ended extraction does not. Since the benchmark uses catalog-generated queries with high lexical overlap, we argue that the pipeline’s semantic robustness positions it for stronger generalization to real-world scenarios.

Resumen

Los catálogos paramétricos—donde los ítems comparten un concepto pero difieren en valores discretos de parámetros—son habituales en construcción, fabricación y comercio electrónico. Los métodos de recuperación estándar tratan los ítems como texto plano, provocando lo que denominamos *colapso paramétrico*: los modelos densos alcanzan un 96–99% de precisión a nivel de concepto, pero solo un 12–45% a nivel de ítem. Proponemos un pipeline de tres etapas: (1) identificación del concepto mediante recuperación densa, (2) extracción de parámetros mediante reglas o LLM, y (3) búsqueda determinista en el catálogo. Sobre el catálogo BC3CAT (47.500 ítems, 16.590 consultas), el pipeline basado en reglas alcanza un 90,3% de Acc@1, superando a todos los métodos neuronales y al BM25 estándar (86,9%). La ablación con dos LLMs y cinco estrategias de prompt muestra que la clasificación cerrada se beneficia del escalado de modelo, mientras que la extracción abierta no.

Keywords: information retrieval, parametric catalogs, construction NLP, structure-aware retrieval, large language models, slot-filling

Palabras clave: recuperación de información, catálogos paramétricos, PLN en construcción, recuperación estructurada, modelos de lenguaje grandes, extracción de slots

1 Introduction

Large infrastructure projects rely on parametric construction catalogs containing tens of thousands of work items, where accurately retrieving the correct entry from project documentation is critical for cost estimation, procurement, and budget control [9, 17]. These catalogs follow a common organizational pattern: each item is defined by a *concept* (a base template describing a type of work) combined with a set of *parameter values* (discrete attributes such as material type, dimensions, terrain conditions, and execution specifications). A single concept may expand into

hundreds or thousands of parametric variants, all sharing nearly identical textual descriptions but differing in small—yet cost-critical—parameter values.

This paper addresses the retrieval problem on such catalogs: given a natural-language description of a work item, retrieve the exact matching entry from a catalog of nearly 40,000 candidates. The challenge is compounded by terminology variability across stakeholders—field workers, designers, and cost estimators often use different terms for identical concepts [5, 12]—and by the critical role of numeric parameters, where even minor discrepancies lead to costly errors in project estimation [17].

Benchmark characteristics and implications. A key aspect of this work that must be stated upfront concerns the nature of the evaluation queries. The BC3CAT benchmark, derived from ADIF’s parametric price catalog for Spanish railway infrastructure, uses catalog-generated short descriptions (*resumen*) as queries against catalog-generated long descriptions (*texto*). Both are produced from the same underlying parametric templates, resulting in high lexical overlap between query and target. This design provides the closest possible approximation to a perfect retrieval scenario—an upper-bound evaluation setting where every query has an unambiguous correct answer and the query text is maximally informative.

This benchmark property has a direct consequence: lexical methods such as BM25 can exploit exact token matching against the high lexical overlap, achieving 97.4% item-level accuracy with parameter-aware tokenization (Section 3). However, in real-world deployment—cost estimators drafting specifications with company-specific wording, field engineers writing abbreviated progress reports—this lexical overlap is substantially reduced. Standard BM25 without parameter-aware tokens already drops to 86.9% even on this favorable benchmark, and realistic phrasing variation would erode performance further.

Neural methods, by contrast, are designed to tolerate exactly this kind of phrasing variation. Yet they exhibit a systematic failure that we term *parametric collapse*: dense retrieval models achieve 96–99% accuracy at the concept (parent) level but only 12–45% at the item level, because embeddings map parametric variants—items that share >95% of their text—into near-identical vector representations. This collapse is not a limitation of any specific model but a structural mismatch between flat-text retrieval and the internal organization of parametric catalogs.

Our approach. We observe that the retrieval problem decomposes naturally into two subproblems with very different difficulty profiles: *concept identification* (already solved by neural models at 96–99%) and *parameter resolution* (the actual bottleneck, currently unaddressed by standard methods). Based on this observation, we propose a three-stage structure-aware pipeline that separates concept retrieval from parameter resolution, matching the natural structure of the catalog.

Contributions. Our contributions are fourfold:

1. We identify and empirically characterize the *parametric collapse* failure mode, where dense retrieval achieves near-perfect concept-level accuracy but fails at item-level disambiguation.
2. We propose a three-stage pipeline—dense concept retrieval, parameter extraction (rule-based or LLM), and deterministic catalog lookup—that explicitly targets the concept-parameter decomposition.
3. We ablate Stage 2 across two LLM models (Llama 3.1 8B, Phi-4 14B) and five prompt strategies, revealing that task framing (closed-set vs. open-ended) determines whether model scaling helps.
4. We evaluate on the BC3CAT benchmark (47,508 items, 16,590 queries), providing the first results on structure-aware retrieval for parametric construction catalogs.

2 Related Work

Construction catalog retrieval. The application of NLP to construction document processing has evolved from rule-based and bag-of-words methods [2] through support vector machines [10] to transformer-based approaches [14, 7]. Systematic reviews document this progression [16, 5]. However, most work addresses category-level classification rather than item-level retrieval, and classification accuracy degrades as taxonomic granularity increases: [2] reported 95.9% at top-level divisions but substantially lower for subcategories.

A recent systematic comparison of lexical, neural, and hybrid retrieval methods on a large-scale Spanish construction catalog (under review) established that optimized BM25 achieves 97.4% item-level Acc@1, while the best neural method (BGE-M3 ColBERT) reaches only 44.8%. That work identified numeric mismatches as the dominant error source (~98% of neural failures) but did not propose structure-aware solutions. The present study builds on those findings by targeting the parametric collapse phenomenon with a decomposed pipeline.

Structure-aware retrieval in product search. In e-commerce, several works address the structure of product catalogs. [4] developed structured matching modules encoding multi-field product information into unified representations. [3] demonstrated that sparse TF-IDF with extreme multi-label classification (PECOS) outperformed dense embeddings on catalogs exceeding 100 million items. [6] proposed a two-stage architecture for procurement matching: dual transformer encoders followed by syntactic reranking. These approaches recognize catalog structure but do not decompose retrieval into concept identification and parameter resolution as distinct stages.

LLMs for structured information extraction. Large language models have demonstrated strong zero-shot capabilities for structured extraction tasks, including named entity recognition, slot-filling, and form understanding. Recent open-source models in the 7–14B parameter range, such as Llama 3.1 [8] and Phi-4 [1], can perform schema-guided extraction when provided with explicit output constraints. Our Stage 2 leverages this capability by framing parameter extraction as constrained slot-filling over a small discrete label space.

3 The Parametric Collapse Problem

3.1 Problem Formulation

In parametric catalogs, each item d can be decomposed as:

$$d = (c, \mathbf{p}), \quad \mathbf{p} = (p_1, p_2, \dots, p_K) \quad (1)$$

where c is the *concept* (shared base template) and $\mathbf{p}$ is a vector of K parameter values drawn from discrete, finite domains $p_k \in \mathcal{V}_k$. All items sharing the same concept c form a *concept group*; within a group, items differ only in their parameter combination $\mathbf{p}$.

Standard retrieval methods encode each item as a flat text representation $\text{enc}(d)$, ignoring the $(c, \mathbf{p})$ decomposition. When items within a concept group share >95% of their text—differing only in parameter value mentions—flat-text encodings produce near-identical representations:

$$\|\text{enc}(d_i) - \text{enc}(d_j)\| \approx 0 \quad \text{for } d_i, d_j \in \text{group}(c) \quad (2)$$

This *parametric collapse* makes item-level discrimination unreliable while leaving concept-level retrieval intact.

Table 1: Parametric collapse: parent-level vs. item-level Acc@1 across retrieval methods on BC3CAT (16,590 queries, 47,508 items). The gap column quantifies the accuracy loss from concept to item level.

Method	Parent Acc@1	Item Acc@1	Gap
BM25 (param-aware tokens)	0.985	0.974	−1.1pp
BM25 (standard unigram)	0.973	0.869	−10.4pp
BGE-M3 ColBERT	0.997	0.448	−54.9pp
BGE-M3 Sparse	0.994	0.125	−86.9pp
BGE-M3 Dense	0.960	0.127	−83.3pp
Dense E5	0.982	0.134	−84.8pp

3.2 Empirical Evidence

Table 1 presents results from a systematic benchmark of retrieval methods on the BC3CAT dataset (under review), evaluated at both parent level (concept group) and item level (exact parametric variant).

Two patterns emerge. First, all neural methods suffer dramatic parent-to-item collapse: they achieve 96–99.7% at concept level but only 12–45% at item level. The root cause is the parametric structure: the largest concept groups contain up to 6,336 variants across 5 parameter axes, and semantic models map these variants into overlapping regions of the embedding space.

Second, BM25 with parameter-aware tokenization achieves 97.4% item-level accuracy by exploiting the high lexical overlap between catalog-generated queries and targets. However, this result depends on the benchmark’s favorable conditions: queries generated from the same templates as targets. Standard BM25 without parameter-aware tokens already drops 10.5 percentage points, and real-world phrasing variation—with abbreviations, synonyms, and implicit parameters—would further reduce lexical overlap.

3.3 The Decomposition Insight

The parent-level results reveal that concept retrieval is essentially solved: neural methods achieve 96–99% accuracy at identifying the correct concept group. The actual bottleneck is *parameter resolution*—disambiguating among parametric variants within the identified group. This decomposition suggests a pipeline architecture that addresses each sub-problem with the most appropriate technique.

4 Proposed Method: Three-Stage Pipeline

We propose a structure-aware retrieval pipeline that separates concept identification from parameter resolution through three stages (Figure 1).

4.1 Stage 1: Dense Concept Retrieval

Stage 1 uses the E5 multilingual model (`intfloat/multilingual-e5-base`, 278M parameters, 768 dimensions) [15] to identify the concept group. Rather than encoding concept-level texts directly (which proved insufficient due to near-identical concept descriptions across tube-diameter groups), we perform item-level retrieval and derive the parent key from the top-1 result via a precomputed item-to-concept mapping. This approach achieves 98.2% parent-level Acc@1 on the benchmark.

E5 was selected as the lightest model among those benchmarked while maintaining strong parent-level performance. The existing dense index (47,514 vectors) is reused without modification.

Input: Natural-language query q Stage 1 — Concept Retrieval (Dense E5) Encode q with E5; retrieve top-1 item from dense index Derive parent_key from item $\rightarrow$ concept group mapping Output: concept group c, candidate set $\mathcal{C}_c$ Stage 2 — Parameter Extraction Given q and schema $\mathcal{S}_c = \{(k, \mathcal{V}_k)\}_{k=1}^K$: Extract $\hat{\mathbf{p}} = (\hat{p}_1, \dots, \hat{p}_K)$ where $\hat{p}_k \in \mathcal{V}_k \cup \{\text{null}\}$ Methods: rule-based matching <i>or</i> LLM slot-filling Stage 3 — Deterministic Catalog Lookup Match $\hat{\mathbf{p}}$ against $\mathcal{C}_c$ Full match $\rightarrow$ single item; partial $\rightarrow$ ranked sub-group Output: Ranked list of items

Figure 1: Three-stage structure-aware retrieval pipeline.

4.2 Stage 2: Parameter Extraction

Given the query q and the concept schema $\mathcal{S}_c$ (axis labels with their discrete value sets), Stage 2 extracts parameter values through structured slot-filling. We evaluate two approaches.

Rule-based extraction. A deterministic extractor using normalized substring matching with longest-match-wins and subsumption filtering for text axes, and context-aware regex matching for numeric axes. This method exploits the fact that parameter values in the BC3CAT benchmark appear as literal substrings in query text, achieving high accuracy at negligible computational cost ($\sim 0.015\text{ms/query}$).

LLM-based extraction. A zero-shot slot-filling approach using local LLMs served via Ollama [11]. The query and concept schema are presented to the LLM, which returns a JSON dictionary mapping axis labels to extracted values. We evaluate two models—Llama 3.1 8B [8] (4.9 GB) and Phi-4 14B [1] (9.1 GB, Microsoft)—across five prompt strategies:

- **Extract:** open-ended extraction (“extract the value for each axis”)
- **Classify:** closed-set selection with pipe-separated options (“classify the query into one of these values”)
- **Twostep:** free-form extraction followed by schema-guided matching (two LLM calls)
- **Paraaware:** per-axis detection followed by value matching, with full value lists shown (two LLM calls)
- **Paraaware2:** per-axis detection with value lists shown in a single call per axis, returning bare values (N LLM calls, one per axis)

All prompts are in Spanish to match the catalog language, with temperature 0.0 for reproducibility.

4.3 Stage 3: Deterministic Catalog Lookup

Stage 3 matches the extracted parameter vector $\hat{\mathbf{p}}$ against the candidate set $\mathcal{C}_c$. The parameter space is fully enumerated; given correct extraction, the match is exact and lossless. If one or more axes are unresolved (**null**), the stage returns the sub-group of items consistent with the resolved axes, ranked by a three-tier scoring scheme: matched items (score ~ 1.0), remaining concept group items (score ~ 0.5), and E5 fallback results (score < 0.5).

Table 2: Evaluation conditions. Pipeline uses E5 for Stage 1; oracle uses ground truth.

Condition	Stage 1	Stage 2
Rules (pipeline)	E5 retrieval	Rule-based
Rules (oracle)	Ground truth	Rule-based
Phi-4 classify (pipeline)	E5 retrieval	Phi-4 14B (classify)
Phi-4 classify (oracle)	Ground truth	Phi-4 14B (classify)
Llama extract (pipeline)	E5 retrieval	Llama 3.1 8B (extract)
Llama extract (oracle)	Ground truth	Llama 3.1 8B (extract)

5 Experimental Setup

5.1 Dataset

The BC3CAT dataset derives from ADIF’s parametric price catalog for Spanish railway infrastructure. We focus on the OEB subcategory (platform system infrastructure), which contains 47,508 distinct items organized into 25 concept groups. Group sizes range from 3 items (1 axis) to 6,336 items (5 axes with 3–8 values each). Each item has a short-format description (*resumen*, mean 19.7 words) used as the query and a long-format description (*texto*, mean 84.5 words) used as the retrieval target.

As noted in Section 1, both descriptions are generated from the same parametric templates, producing high lexical overlap between query and target. This represents an upper-bound evaluation setting: results on this benchmark are optimistic estimates of real-world performance, particularly for lexical methods.

5.2 Evaluation Protocol

We evaluate 16,590 queries (99% confidence, $\pm 1\%$ margin) against the full corpus of 47,508 items. The primary metric is item-level Acc@1; we also report parent-level Acc@1, MRR, and Recall@10.

Pipeline vs. oracle conditions. To isolate the contribution of each stage, we evaluate both *pipeline* conditions (Stage 1 uses E5 retrieval) and *oracle* conditions (Stage 1 is given the ground-truth concept group). The pipeline–oracle gap quantifies the impact of Stage 1 errors.

Evaluation conditions. We report six primary conditions:

These represent the best-performing configuration for each Stage 2 approach, selected from the full 2 models $\times$ 4 prompts exploration (Section 6.4).

5.3 Baselines

We compare against baselines from a systematic retrieval benchmark on the same dataset (under review): BM25 with parameter-aware tokens (97.4%), BM25 standard unigram (86.9%), BGE-M3 ColBERT (44.8%), Dense E5 standalone (13.4%), and hybrid fusion methods.

5.4 Implementation

All experiments use Python 3.10. Stage 1 reuses the existing E5 index built with sentence-transformers v2.2.2 [13]. Stage 2 LLM inference uses Ollama 0.17.7 (Docker, GPU-enabled) with pinned model versions for reproducibility. Stage 3 lookup uses a precomputed reverse index from the parquet data. Evaluation runs on servers with Intel i9-13900KF CPUs and NVIDIA RTX 4090 GPUs. The full evaluation of 16,590 queries takes 10–50 minutes for the rules pipeline and 3–5 hours for LLM conditions. Code is available at [repository URL upon acceptance].

Table 3: Item-level performance on the BC3CAT benchmark (16,590 queries). Baselines from prior work on this dataset (under review); pipeline conditions from this work.

Method	Acc@1	Parent Acc@1	MRR	Type
<i>Baselines (flat retrieval)</i>				
BM25 param-aware	0.974	0.985	0.979	Lexical
BM25 standard	0.869	0.973	0.898	Lexical
3-way fusion	0.890	—	0.908	Hybrid
BGE-M3 ColBERT	0.448	0.997	0.580	Neural
Dense E5	0.134	0.982	0.251	Neural
<i>Structure-aware pipeline (this work)</i>				
Rules (pipeline)	0.903	0.985	—	Pipeline
Rules (oracle)	0.914	1.000	—	Pipeline
Phi-4 classify (pipeline)	0.877	0.985	—	Pipeline
Phi-4 classify (oracle)	0.886	1.000	—	Pipeline
Llama extract (pipeline)	0.809	0.985	—	Pipeline
Llama extract (oracle)	0.821	1.000	—	Pipeline

6 Results

6.1 Main Results

Table 3 presents item-level performance across all pipeline conditions and baselines.

The rules-based pipeline achieves 90.3% item-level Acc@1, substantially outperforming all standalone neural methods (best: BGE-M3 ColBERT at 44.8%) and the hybrid fusion approach (89.0%). The pipeline also surpasses standard BM25 (86.9%) by 3.4 percentage points, demonstrating that the structure-aware decomposition provides measurable gains even on this lexical-overlap-favorable benchmark.

BM25 with parameter-aware tokenization remains the top performer at 97.4%, reflecting its ability to exploit the exact lexical overlap present in catalog-generated queries. The 7.1 percentage point gap between BM25 param-aware and the rules pipeline stems primarily from Stage 2 extraction failures on a single parameter axis (Section 6.5).

6.2 Stage 1 Is Not the Bottleneck

The pipeline-oracle gap is remarkably small across all conditions: 1.1% for rules, 0.9% for Phi-4 classify, and 1.2% for Llama extract. This confirms that E5 concept retrieval (98.5% parent Acc@1) is not the bottleneck—the dominant error source is Stage 2 parameter extraction.

All three pipeline conditions share the same 252 Stage 1 errors (1.5% of queries), exclusively involving concept confusions between similar groups (e.g., 110mm vs. 160mm polyethylene tube canalizations, or manual vs. machine trench excavation).

6.3 Stage 2: Rules vs. LLMs

Table 4 compares Stage 2 methods under oracle conditions (perfect concept retrieval), isolating extraction quality.

The rule-based extractor outperforms both LLM conditions while being approximately 40,000–60,000× faster. This is expected on the current benchmark: since queries are generated from catalog templates, parameter values appear as literal substrings in the query text, making rule-based matching highly effective. LLM extraction introduces errors through value normalization failures (e.g., returning 1.10 instead of the catalog value 1,10 m) and omissions (returning null for axes that are present in the query).

Table 4: Stage 2 comparison under oracle conditions (16,590 queries). Rules never produce wrong values—only null when unable to match.

Stage 2 Method	Item Acc@1	Errors	Time/query
Rule-based	0.914	1,434	0.015ms
Phi-4 14B (classify)	0.886	1,887	~886ms
Llama 3.1 8B (extract)	0.821	2,976	~585ms

Table 5: Per-axis accuracy across prompt strategies and models (20-query development sample, 74 total axis extractions). Rule-based included for reference.

Prompt Strategy	Llama 3.1 8B	Phi-4 14B
Rule-based	100.0%	—
Extract	87.8%	77.0%
Classify	86.5%	100.0%
Paraaware2	82.4%	93.2%
Twostep	64.9%	58.1%
Paraaware	48.6%	78.4%

Critically, the rule-based extractor never produces wrong values—it either matches correctly or returns `null`. This predictable failure mode makes it a reliable and debuggable baseline.

6.4 Prompt Strategy and Model Scaling

We evaluated all five prompt strategies across both LLM models during development (20-query Stage 2 comparison). Table 5 summarizes the results.

The most striking finding is the highly non-linear interaction between prompt strategy and model size. For *classify* (closed-set selection from pipe-separated options), Phi-4 achieves perfect accuracy while Llama reaches only 86.5%—a 13.5 percentage point improvement from model scaling. For *extract* (open-ended value extraction), Phi-4 actually *degrades* compared to Llama (77.0% vs. 87.8%), as the larger model produces more selective responses that increase omissions.

The *paraaware2* strategy—which makes N separate LLM calls (one per axis), each showing the full list of possible values—dramatically improves over the original *paraaware* for both models: Llama improves from 48.6% to 82.4% (+33.8pp) and Phi-4 from 78.4% to 93.2% (+14.8pp). Showing possible values upfront eliminates the abstract detection failures that plagued the original two-step approaches. However, *paraaware2* still does not surpass the simpler single-call *classify* mode, which remains more accurate (especially for Phi-4) while requiring only one LLM call regardless of the number of axes.

This pattern generalizes: closed-set tasks (*classify*, *paraaware2*) benefit from model scaling because they require reasoning over a constrained option space, while open-ended extraction tasks do not benefit because the difficulty lies in matching free-form text to exact catalog values—a normalization problem, not a reasoning problem.

The two-step approaches (*twostep*, *paraaware*) consistently underperform single-call methods, as Step 1 produces verbose compound descriptions that Step 2 cannot reliably map to individual schema values.

6.5 Error Analysis

We classified all failed queries across the six primary conditions. Errors fall into two categories:

E1 — Wrong concept (Stage 1 error). 252 queries (15.7% of rules pipeline errors) where E5 retrieves the wrong concept group. These are exclusively confusions between similar groups

Table 6: Per-axis extraction accuracy (rules, oracle condition, 16,590 queries). Only the three axes with <100% accuracy are shown; 10 other axes achieve perfect accuracy.

Axis	Occurrences	Null extractions	Accuracy
TRABAJO	16,571	1,017	93.9%
CONDICIONES DE EJEC.	16,580	34	99.8%
BANDA DE MANT.	16,571	28	99.8%

(e.g., polyethylene 50mm vs. 160mm, manual vs. machine excavation). Notably, these errors are themselves a manifestation of parametric collapse at the concept level: the concept descriptions differ by a single word or number (e.g., “110mm” vs. “160mm”, “a mano” vs. “a máquina”), causing dense embeddings to map distinct concepts into overlapping regions of the vector space.

E2 — Parameter extraction failure (Stage 2 error). 1,434 queries (100% of rules oracle errors) where at least one axis is unresolved. For the rule-based extractor, all errors are partial extractions (null on one or more axes); zero wrong-value errors were observed.

Per-axis analysis. Table 6 reveals that errors are concentrated on a single axis.

The TRABAJO (work shift) axis accounts for approximately 70% of all rules oracle errors (1,017 out of 1,434 null extractions). This axis encodes compound values combining shift type, maintenance window, and execution conditions into a single textual string, making substring matching unreliable. The remaining axes—including numeric axes like N° TUBOS, TIPO DE TERRENO, and DIÁMETROS—achieve perfect extraction accuracy.

Hardest concept groups. Error rates vary substantially across concept groups: OEB190\$ (manual trench excavation, 35.8% error rate) and OEB200\$ (machine trench excavation, 33.6%) are the most challenging, while the large polyethylene tube groups (OEB030\$, OEB040\$) have error rates below 6.3%.

Rules vs. LLM disagreement. Under oracle conditions, 887 queries fail for rules but succeed for Llama, while 2,429 fail for Llama but succeed for rules. For Phi-4 classify, the split is 804 rules-only failures vs. 1,257 Phi-4-only failures. This partial complementarity suggests that combining rules and LLMs could yield further improvements.

7 Discussion

Why rules outperform LLMs on this benchmark. The rule-based extractor’s dominance is a direct consequence of the benchmark’s catalog-generated queries: parameter values appear as literal substrings, making pattern matching nearly optimal. LLMs introduce errors through value normalization (e.g., reformatting decimal separators, abbreviating units) and omissions—problems that arise precisely because the LLM attempts to *understand* the text rather than match it verbatim. On realistic queries with paraphrased parameters, this relationship would likely invert: LLMs’ ability to recognize “triple tubing” as “3 tubos” or “nighttime” as “Nocturno” would provide an advantage that rules cannot replicate.

The pipeline vs. BM25. The structure-aware pipeline (90.3%) does not surpass BM25 with parameter-aware tokenization (97.4%) on this benchmark. This is expected: BM25 param-aware tokens exploit the high lexical overlap that characterizes catalog-generated queries, achieving near-perfect performance on this specific evaluation setting. However, the pipeline has a fundamentally different generalization profile. Its Stage 1 (dense E5) is robust to phrasing variation by

design, and its Stage 2 explicitly reasons over the parameter schema rather than relying on lexical overlap. When lexical overlap diminishes—as it inevitably does with real-world queries—we expect the pipeline to degrade more gracefully than BM25.

Practical implications. For deployment on catalog-generated or template-based queries, BM25 with domain-specific tokenization remains the clear choice: it is fast, accurate, and requires no GPU infrastructure. For deployment with heterogeneous real-world queries—where phrasing varies across regions, companies, and individual users—the three-stage pipeline offers a principled architecture that combines semantic robustness with explicit parameter handling. The rules-based variant requires no GPU and runs in milliseconds; the LLM variant offers potential for handling paraphrased parameters at the cost of latency and infrastructure.

Limitations. This study has several limitations. First, we evaluate on a single catalog chapter (OEB) with homogeneous parametric structure; generalization to other chapters and catalogs requires validation. Second, and most critically, all evaluation queries are catalog-generated, which inflates lexical methods’ performance and limits conclusions about real-world generalization. Third, the LLM evaluation uses only two models; larger models or fine-tuned variants may close the gap with rules. Fourth, the pipeline’s three-tier ranking for partial matches has not been optimized.

8 Conclusions and Future Work

We identified parametric collapse—the systematic failure of dense retrieval models to disambiguate parametric variants in structured catalogs—and proposed a three-stage pipeline that decomposes retrieval into concept identification and parameter resolution. On the BC3CAT benchmark, the rule-based pipeline achieves 90.3% item-level Acc@1, outperforming all neural baselines and surpassing standard BM25 by 3.4 percentage points, while approaching the domain-tuned BM25 upper bound (97.4%). Error analysis reveals that 70% of residual failures stem from a single parameter axis, indicating a clear and bounded path for improvement.

The most important direction for future work is the construction of a *realistic query dataset* with human-authored queries—cost estimator paraphrases, field engineer notes, abbreviated descriptions—to evaluate whether the pipeline’s combination of semantic retrieval and explicit parameter resolution generalizes as expected beyond the catalog-generated benchmark. Additional future work includes lightweight classifiers (e.g., per-axis BERT heads) as efficient alternatives to LLM-based extraction, cross-domain evaluation on other parametric catalogs (e-commerce, industrial parts), and the integration of retrieval pipelines into BIM and estimation software for semi-automated construction workflows.

Acknowledgments

The authors acknowledge the use of generative AI tools to assist in drafting sections of the manuscript and producing code. All scientific interpretations, data analyses, and final revisions were performed by the authors.

References

- [1] Marah Abdin, Jyoti Aneja, Hany Awadalla, et al. Phi-4 technical report. *arXiv preprint arXiv:2412.08905*, 2024. Microsoft Research.

- [2] Carlos H. Caldas and Lucio Soibelman. Automating hierarchical document classification for construction management information systems. *Automation in Construction*, 12(4):395–406, July 2003.
- [3] Wei-Cheng Chang, Daniel Jiang, Hsiang-Fu Yu, Choon Hui Teo, Jiong Zhang, Kai Zhong, Kedarnath Kolluri, Qie Hu, Nikhil Shandilya, Vyacheslav Ievgrafov, Japinder Singh, and Inderjit S. Dhillon. Extreme Multi-label Learning for Semantic Matching in Product Search. In *Proceedings of the 27th ACM SIGKDD Conference on Knowledge Discovery & Data Mining*, pages 2643–2651, Virtual Event Singapore, August 2021. ACM.
- [4] Jason Ingyu Choi, Surya Kallumadi, Bhaskar Mitra, Eugene Agichtein, and Faizan Javed. Semantic Product Search for Matching Structured Product Catalogs in E-Commerce, August 2020.
- [5] Sehwan Chung, Seonghyeon Moon, Junghoon Kim, Jungyeon Kim, Seungmo Lim, and Seokho Chi. Comparing natural language processing (NLP) applications in construction and computer science using preferred reporting items for systematic reviews (PRISMA). *Automation in Construction*, 154:105020, October 2023.
- [6] Washington Cunha, Celso França, Leonardo Rocha, and Marcos André Gonçalves. TPDR: A Novel Two-Step Transformer-based Product and Class Description Match and Retrieval Method, October 2023.
- [7] J. Ignacio Deza, Hisham Ihshaish, and Lamine Mahdjoubi. A Machine Learning Approach to Classifying Construction Cost Documents into the International Construction Measurement Standard, October 2022.
- [8] Aaron Grattafiori, Abhimanyu Dubey, Abhinav Jauhri, et al. The Llama 3 herd of models. *arXiv preprint arXiv:2407.21783*, 2024. Meta AI.
- [9] María Martínez-Rojas, Nicolás Marín, and M. Amparo Vila. An Approach for the Automatic Classification of Work Descriptions in Construction Projects. *Computer-Aided Civil and Infrastructure Engineering*, 30(12):919–934, 2015.
- [10] María Martínez-Rojas, Jose Manuel Soto-Hidalgo, Nicolás Marín, and M. Amparo Vila. Using Classification Techniques for Assigning Work Descriptions to Task Groups on the Basis of Construction Vocabulary. *Computer-Aided Civil and Infrastructure Engineering*, 33(11):966–981, 2018.
- [11] Ollama. Ollama: Run large language models locally. <https://ollama.com>, 2024. Version 0.17.7.
- [12] Luca Rampini and Fulvio Re Cecconi. Artificial intelligence in construction asset management: A review of present status, challenges and future opportunities. *Journal of Information Technology in Construction*, 27:884–913, October 2022.
- [13] Nils Reimers and Iryna Gurevych. Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks, August 2019.
- [14] Shengxian Tang, Hexu Liu, Manea Almatared, Osama Abudayyeh, Zhen Lei, and Alvis Fong. Towards Automated Construction Quantity Take-Off: An Integrated Approach to Information Extraction from Work Descriptions. *Buildings*, 12(3):354, March 2022.
- [15] Liang Wang, Nan Yang, Xiaolong Huang, Binxing Jiao, Linjun Yang, Daxin Jiang, Rangan Majumder, and Furu Wei. Text Embeddings by Weakly-Supervised Contrastive Pre-training, February 2024.

- [16] Chengke Wu, Xiao Li, Yuanjun Guo, Jun Wang, Zengle Ren, Meng Wang, and Zhile Yang. Natural language processing for smart construction: Current status and future directions. *Automation in Construction*, 134:104059, February 2022.
- [17] Chengke Wu, Xin Li, Yuanjun Guo, Jun Wang, Zengle Ren, Meng Wang, and Zhiling Yang. Natural language processing for smart construction: Current status and future directions. *Automation in Construction*, 2022.